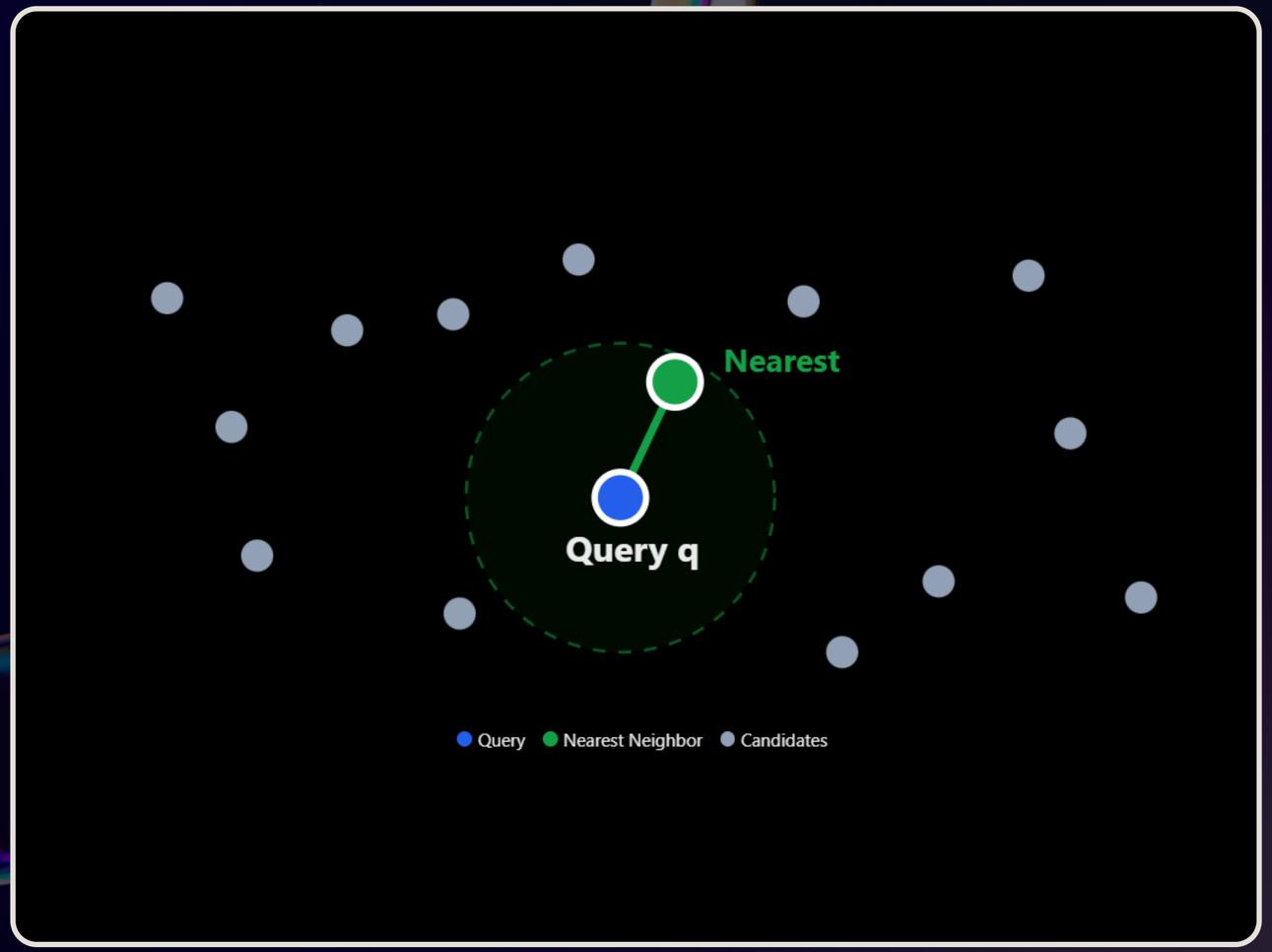
Project GPU-Accelerated Exact Nearest Neighbor Search

Using CUDA

Department of Computer Science Kasetsart University · November 9, 2026 Presented by: Thanaphat 6710450929

Exact Nearest Neighbor Search



Problem:

ค้นหา Vector ที่ใกล้ Query มากที่สุดจาก Candidates จำนวน N ตัว

Exact Search:

ตรวจสอบทุก Candidate เพื่อให้ได้ผลลัพธ์ที่ตรงตามนิยาม ไม่ใช้ Approximation

Motivation:

การคำนวณ Distance ของแต่ละ Candidate เป็นอิสระต่อกัน จึงเหมาะกับ Data Parallelism

Challenge:

GPU มี Parallelism สูง แต่มีต้นทุน Memory Transfer และการจัดการข้อมูล

How Exact Nearest Neighbor Search Works

Squared Euclidean Distance ($L2^2$)

$$d_i = \sum_{j=0}^{D-1} (x_i[j] - q[j])^2$$

q = Query Vector

x_i = Candidate Vector ตัวที่ i

N = จำนวน Candidates

D = จำนวน Dimensions

d_i = ระยะทางยกกำลังสองระหว่าง Query กับ Candidate ตัวที่ i

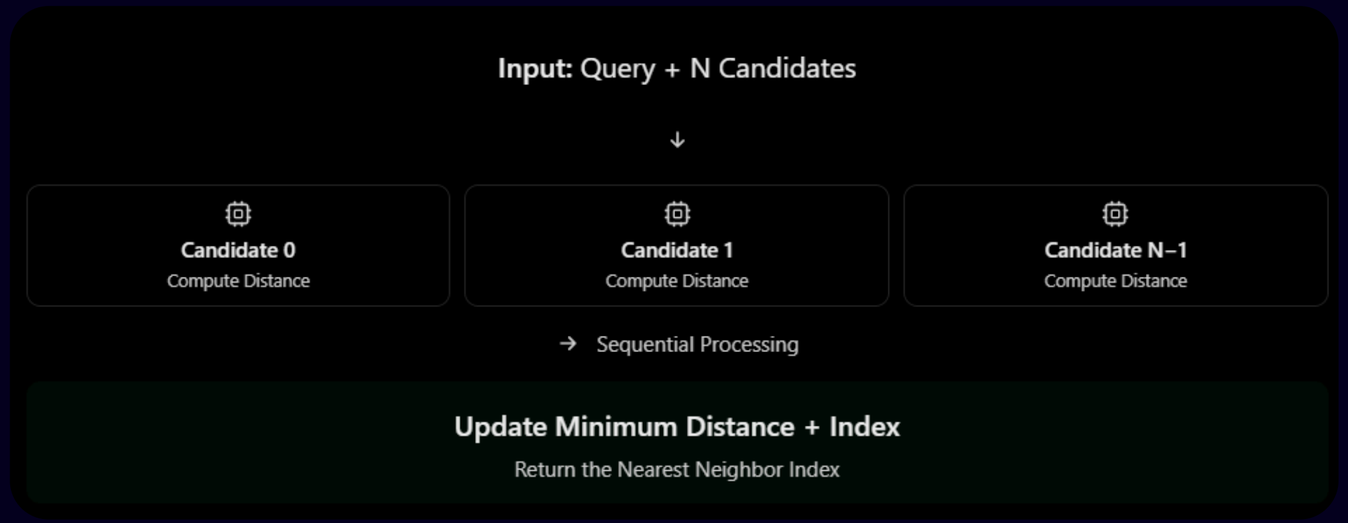
Input: Query + N Candidates

Compute Distance for Every Candidate

Find Minimum Distance

Output: Nearest Neighbor Index

CPU Baseline

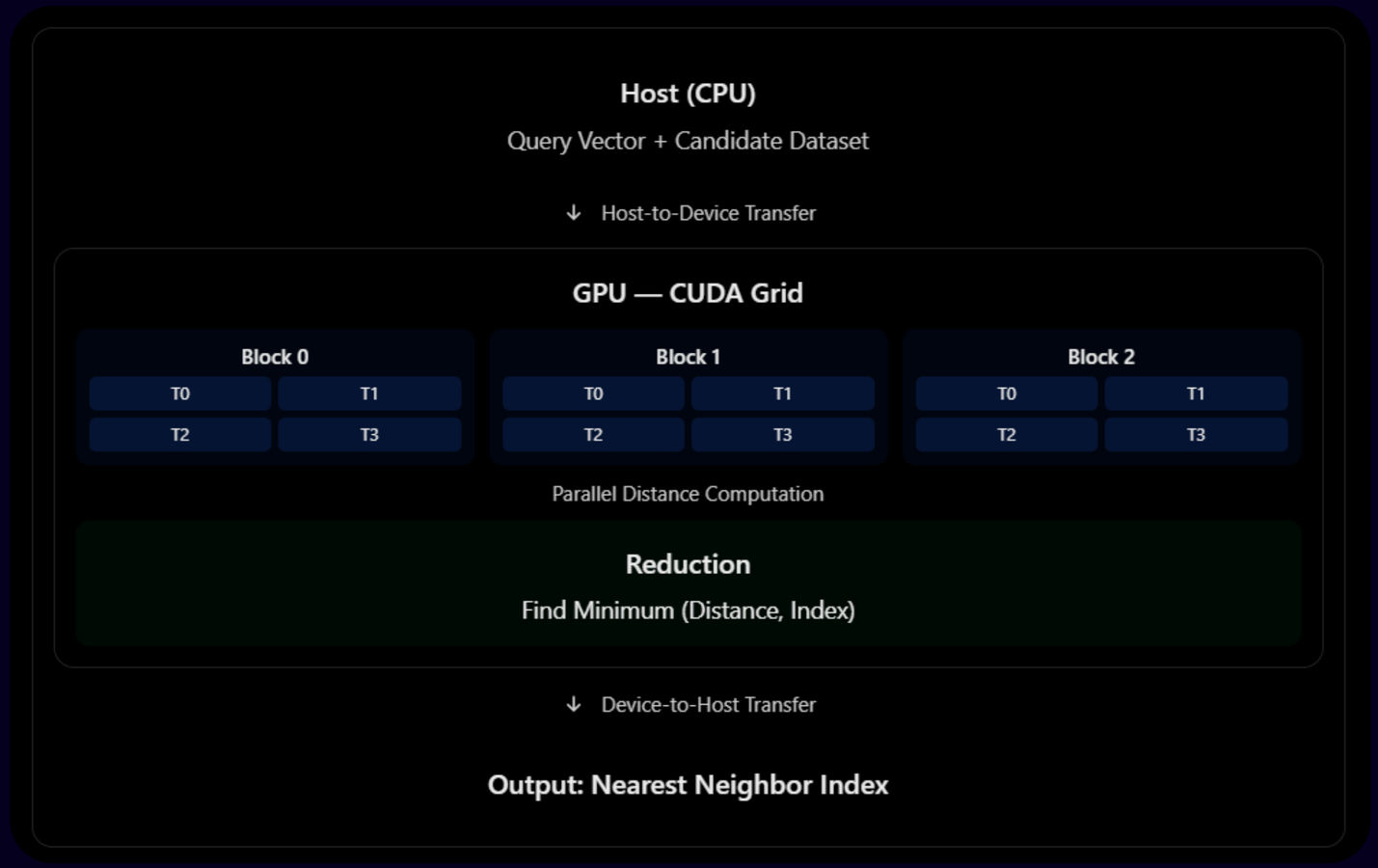


Purpose of CPU Baseline

- Correctness: ใช้ตรวจสอบความสอดคล้องของผลลัพธ์กับ CUDA V0–V3
- Performance: ใช้เป็น Baseline สำหรับเปรียบเทียบ GPU Raw End-to-End Latency

- **Exhaustive Search: ตรวจสอบ Candidates ทุกตัว**
- **Sequential Execution: คำนวณทีละ Candidate**
- **Time Complexity: O(ND)**
- **Precision: Float32**
- **Tie-breaking: เลือก Index น้อยที่สุดเมื่อระยะทางเท่ากัน**

CUDA Architecture



CUDA V0–V3: Implementation Evolution

V.0

ใช้ AoS ในการจัดเก็บข้อมูล โดยให้แต่ละ CUDA Thread คำนวณระยะทางของ Candidate และใช้ Serial Device Reduction เพื่อค้นหาค่าต่ำสุด เป็นเวอร์ชันพื้นฐานสำหรับเปรียบเทียบประสิทธิภาพ

V.1

ปรับ Memory Layout จาก AoS เป็น SoA เพื่อให้การอ่านข้อมูลของ GPU Threads มีความต่อเนื่องมากขึ้น (Memory Coalescing) แต่ยังคงใช้ Serial Device Reduction เช่นเดียวกับ V0

V.2

พัฒนาจาก V1 โดยเปลี่ยน Serial Reduction เป็น Hierarchical Parallel Reduction ใช้ Shared Memory รวมผลลัพธ์ภายในแต่ละ Block เพื่อลดการประมวลผลแบบลำดับและเพิ่ม Parallelism

V.3

ปรับปรุง V2 ด้วย Warp Shuffle ให้ Threads ภายใน Warp แลกเปลี่ยนข้อมูลผ่าน Registers เพื่อลดการใช้ Shared Memory และ Synchronization บางส่วน โดยยังคงใช้ Hierarchical Reduction

Parallel Reduction Strategies

Serial Reduction (V0–V1)

- ใช้ Thread เดียวค้นหา Minimum Distance จาก Candidates ทั้งหมด
- Limitation: Sequential Bottleneck

Shared-Memory Reduction (V2)

- ใช้หลาย Threads คำนวณ Minimum ภายในแต่ละ Block
- ใช้ Shared Memory และ Hierarchical Reduction
- Benefit: เพิ่ม Parallelism และลด Serial Bottleneck

Warp-Shuffle Reduction (V3)

- ใช้ Warp Shuffle เพื่อแลกเปลี่ยนข้อมูลระหว่าง Threads ภายใน Warp
- ลดการใช้ Shared Memory และ Synchronization บางส่วน
- Benefit: ลด Communication Overhead ภายใน Warp

Complexity Comparison :

Strategy	Work	Span
Serial Reduction	O(N)	O(N)
Parallel Reduction Tree	O(N)	O(log N)

Experimental Setup

1. Hardware & Software

Component	Specification
GPU	NVIDIA GeForce RTX 3060 Laptop
Compute Capability	8.6
CPU	Intel Core i7-11370H
CUDA Toolkit	13.3
Operating System	Windows 11
Compiler	MSVC 19.51
Build Configuration	Release, Fast Math Disabled

2. Dataset Configuration

Parameter	Values
Candidates (N)	1,024 / 16,384 / 262,144 / 1,048,576
Dimensions (D)	8 / 32 / 128
Total Workloads	12 Configurations
Data Precision	Float32
Query Type	Single Query

3. Benchmark Methodology

- Warm-up: 10 Runs
- Measurement: 100 Runs per Configuration
- Metric: Median Execution Time (ms)
- Correctness: ตรวจสอบผลลัพธ์ก่อนยอมรับผลการทดลอง
- Comparison: CPU Baseline vs CUDA V0–V3

Measurement Methodology

Objective: แยกเวลาประมวลผล CUDA Kernel ออกจากเวลาทำงานทั้งหมด เพื่อประเมินประสิทธิภาพ GPU อย่างถูกต้อง

1. CPU Raw End-to-End (CPU-E2E-RAW)

- วัดเวลาตั้งแต่เรียก CPU Search จนได้รับผลลัพธ์
- รวม Validation และ Sequential Distance Computation

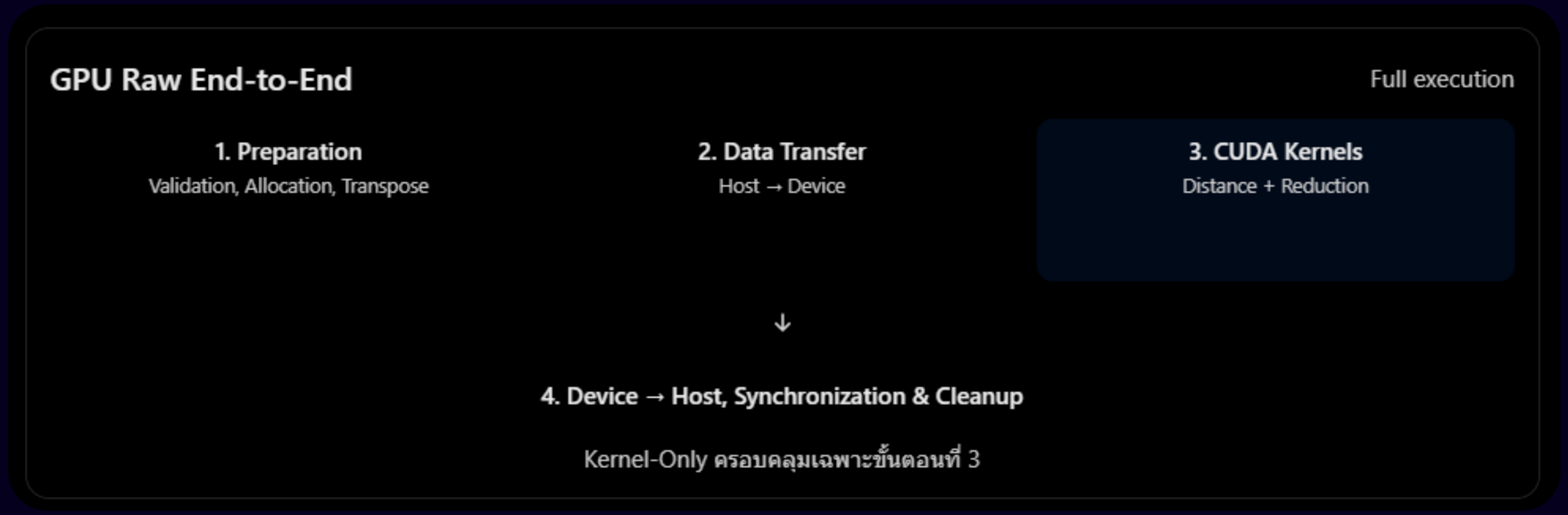
2. GPU Kernel-Only (BND-KERNEL)

- วัดเฉพาะ CUDA Kernel Execution
- รวม Distance Computation และ Reduction Kernels ทั้งหมด
- ไม่รวม Memory Allocation, Data Conversion และ Data Transfer

3. GPU Raw End-to-End (BND-E2E-RAW)

- วัดเวลาตั้งแต่เริ่มเรียก GPU Search จนได้รับผลลัพธ์
- รวม Memory Allocation, AoS-to-SoA Conversion (V1–V3), Host-to-Device Transfer, Kernel Execution, Device-to-Host Transfer และ Synchronization

Performance Measurement Flow



Kernel Speedup ≠ End-to-End Speedup

การที่ CUDA Kernel ทำงานเร็วขึ้น ไม่ได้หมายความว่าโปรแกรม GPU ทั้งระบบจะเร็วกว่า CPU เนื่องจากยังมีต้นทุนการจัดการ Memory และ Data Transfer

CUDA Kernel Performance Comparison

Hypothesis Results

Hypothesis	Description	Result
H2	V1 Kernel faster than V0	REJECTED
H3	V2 Kernel faster than V1	SUPPORTED (12/12)
H4	V3 Kernel faster than V2	REJECTED

Key Performance Results

V2 vs V1 — Median Kernel Speedup

2.233×–84.286×

Speedup range across 12 workloads

V2 Faster than V1

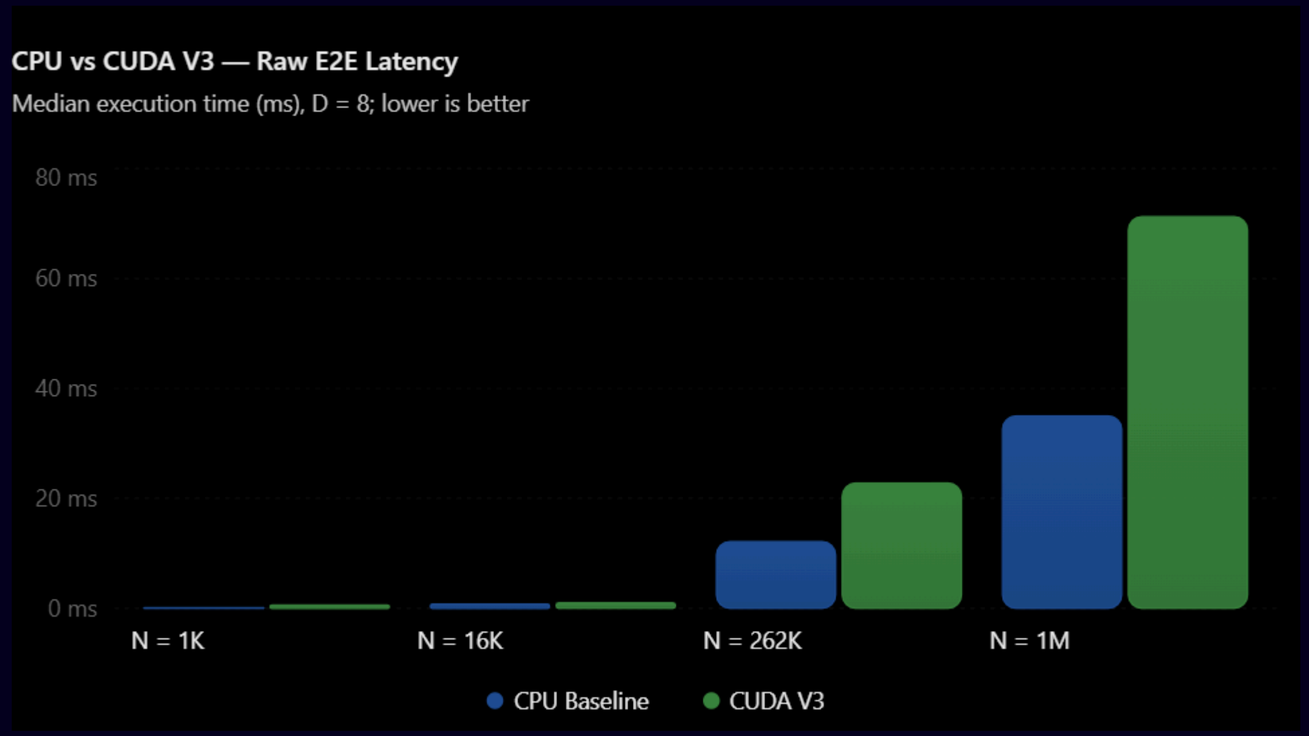
12/12

Workloads supported

Performance Analysis

- V0 → V1: เปลี่ยน AoS เป็น SoA แต่ไม่ได้ทำให้ Kernel เร็วขึ้นทุก Workload
- V1 → V2: Hierarchical Reduction ช่วยลด Serial Bottleneck และเพิ่มประสิทธิภาพ Kernel ทุก Workload
- V2 → V3: Warp Shuffle ไม่ได้ทำให้ Kernel เร็วกว่า Shared-Memory Reduction ทุกกรณี

Benchmark Results



Key Findings

- H1: REJECTED (12/12)
- GPU V3 Raw E2E slower than CPU in all workloads.
- Kernel Speedup ≠ End-to-End Speedup

Performance Bottlenecks

- Memory Allocation
- AoS-to-SoA Transpose
- Host-to-Device Data Transfer
- Synchronization & Cleanup

Cache-Blocked Transpose Optimization

Optimization Technique

Cache-Blocked Transpose

- แบ่งข้อมูลออกเป็น Tiles ขนาดเล็ก เพื่อเพิ่ม Cache Locality
- ปรับปรุงรูปแบบการเข้าถึง Memory ระหว่างการแปลง AoS → SoA
- ทดลอง Tile Sizes: 32×32, 64×32, 64×64

Performance Comparison

Comparison	Supported	Rejected	Inconclusive
Host Transpose	18	18	0
GPU Raw E2E	18	8	10
CPU vs GPU Raw E2E	0	144	0

Experimental Results

Conclusion & Future Improvements

1. Key Achievements

- Correctness: CPU Baseline และ CUDA V0–V3 ผ่านการตรวจสอบความถูกต้อง
- Parallel Reduction: V2 เร็วกว่า V1 ในระดับ Kernel ทั้ง 12 Workloads
- Memory Optimization: Cache Blocking ช่วยปรับปรุงประสิทธิภาพในบาง Workloads
- Performance Evaluation: GPU Raw E2E ยังไม่เร็วกว่า CPU Baseline

2. Project Limitations

- Synthetic Dataset: ยังไม่ได้ทดลองกับ External Public Dataset
- Single Query: ยังไม่มีผลการทดลองสำหรับ Batch Query
- GPU Overhead: มีต้นทุนจาก Memory Allocation, Transpose และ Data Transfer
- Hardware Scope: ทดลองบน RTX 3060 Laptop GPU เพียงเครื่องเดียว

3. Future Work

- Public Dataset: ทดสอบกับข้อมูลจริงจากแหล่งสาธารณะ
- Batch Query Processing: ประมวลผลหลาย Queries เพื่อกระจายต้นทุน Overhead
- Memory Reuse: ใช้ Device-Resident Data และ Persistent Workspace เพื่อลด Allocation และ Data Transfer
- Further Optimization: ศึกษา CUDA V4 และปรับปรุง Kernel เพิ่มเติม

The End